

Context Engine with reusable KV cache cartridges.

THE UNREALISED BENEFIT

Large language models (LLMs) often incorporate multiple text chunks in their inputs to provide the necessary contexts. To speed up the prefill of the long LLM inputs, one can pre-compute the KV cache of a text and re-use the KV cache when the context is reused as the prefix of another LLM input.

However, the reused text chunks are not always the input prefix, which makes precomputed KV caches not directly usable since they ignore the text's cross-attention with the preceding texts. Thus, the benefits of reusing KV caches remain largely unrealized.

$KV(B \mid A) \neq KV(B)$ A chunk cached alone never attended to what precedes it. Concatenating cached chunks is not a full prefill.

WHY UNPOD NEEDS A CORE ENGINE

As we are building backend infrastructure for conversational AI, we need a core engine sitting under the smart context engine, one that not only handles the semantic context but also works on the underlying accessibility of the model inference engine, which eventually impacts the overall performance, latency and cost for each model.

-
- 01 **Performance.** Prefill grows with every chunk of context the agent carries.

 - 02 **Latency.** On a voice turn that prefill sits ahead of the first audio the caller hears.

 - 03 **Cost.** The same context is recomputed for every request, turn and session.

Cartridges fuse per request with selective KV recompute.

THE SEMANTIC SYSTEM ANSWERS

What does this agent need to know about entity A, entity B, and the relationship between them right now? An entity can be a user, an organisation, or anything else.



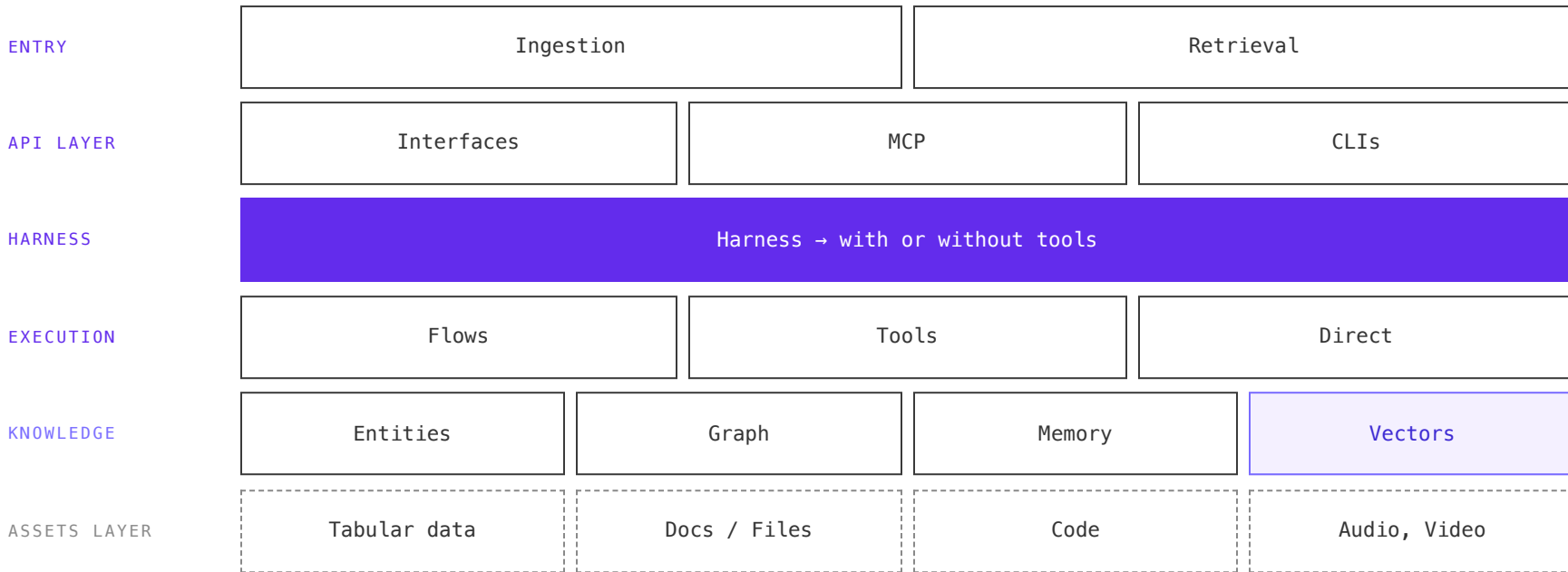
THE INFERENCE SYSTEM ANSWERS

How do we make that knowledge available to the model without repeatedly paying the prefill cost?

PER REQUEST

Each context chunk is compiled into a KV cartridge once and attached on demand, prefix or not. Selective KV recompute updates under 15% of tokens per layer to restore cross-attention, giving full-reuse speed with full-recompute quality (CacheBlend).

Design of the context engine.



WHERE KV LIVES

Entities, graph, memory and vectors are what the Context Compiler compiles into KV cartridges; the assets layer is the source of truth they are derived from.

Problem, outcome, and what we need from NVIDIA.

01 · REFINED PROBLEM

Reused context is rarely the prefix.

Conversational agents carry stable memory about entity A, entity B and their relationship on every turn. Precomputed KV caches only apply when that text is the exact prefix, so today it is re-prefilled per request.

We are building the infrastructure layer for conversational apps, and need a core engine under the semantic context layer that reaches into the inference engine, where performance, latency and cost are decided.

02 · OUTCOME TO DEMONSTRATE

Cartridges beat full prefill on an SLM.

A live voice flow where per-entity context is served from KV cartridges with selective KV recompute, measured against full prefill on the same model and hardware.

TTFT	≥2× lower median time-to-first-token on warm 16–32K context calls.
Cost	Lower GPU time per turn with the update fraction under 15% of tokens per layer.
Quality	Same answer quality as full recompute on conventional voice and messaging flows.

03 · WHAT WE NEED FROM NVIDIA

Engineering access and guidance.

- 01 **Engineering guidance.** On KV cartridge placement across GPU, host and remote tiers, selective recompute inside the inference engine, and where runtime ends and our context engine begins.
- 02 **Cheap GPU access.** Compute for the benchmark and for serving a million-plus users of memory and context at a cost floor that works for SLM plus context.
- 03 **GTM support.** Channels where we can reach developers directly: developer programmes, co-marketing of the benchmark, and a presence in NVIDIA developer ecosystems.